

4.1 인공신경망의 한계와 딥러닝 출현



#01 퍼셉트론의 개념과 역할

퍼셉트론(Perceptron)

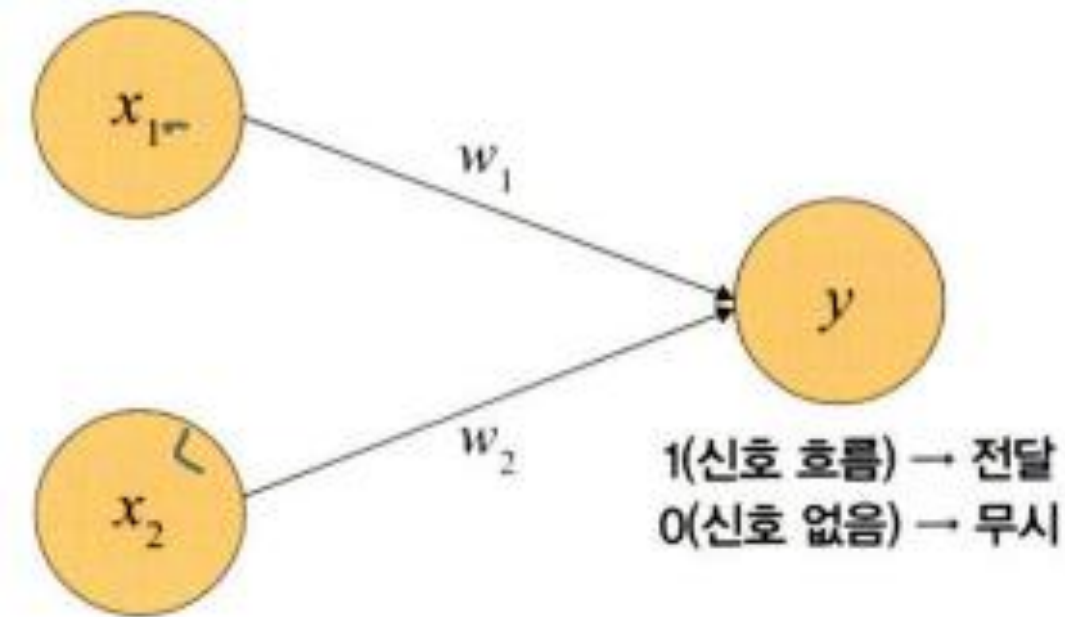
: 여러 입력 신호에 각각 가중치를 적용하고, 이를 종합하여 하나의 출력을 결정하는 인공 신경망의 기본 단위

- 입력 $x \rightarrow$ 가중합 $\rightarrow$ 판단 $\rightarrow$ 출력 y

역할

- 입력 정보의 중요도를 가중치로 반영
- 계산 결과를 기준으로 분류·판단 수행

▼ 그림 4-1 퍼셉트론 원리



#02 단층 퍼셉트론과 AND/OR

단층 퍼셉트론(=선형 분류기)

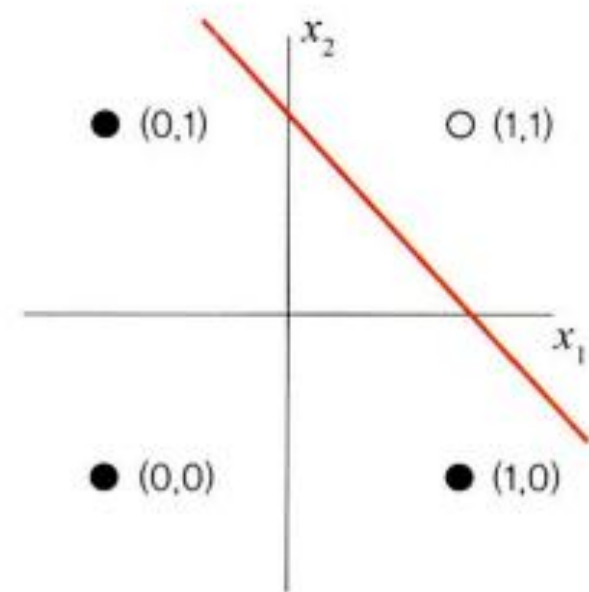
: 하나의 선형 결정 경계로 데이터를 분류

AND 게이트

▼ 표 4-1 AND 게이트

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

▼ 그림 4-2 AND 게이트

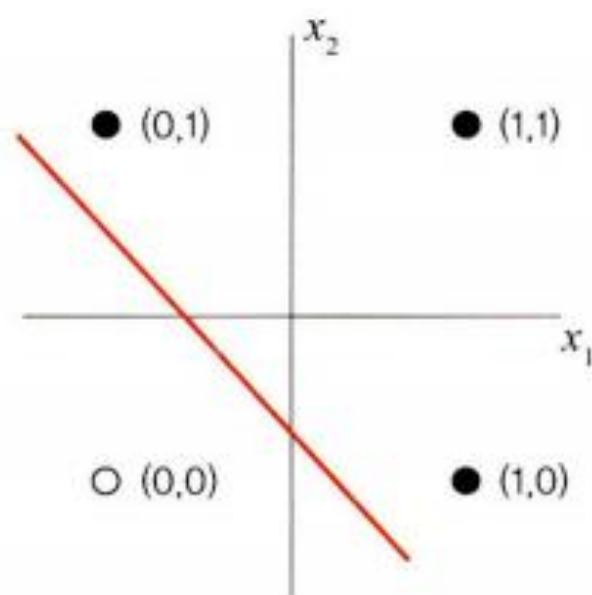


OR 게이트

▼ 표 4-2 OR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

▼ 그림 4-3 OR 게이트



AND / OR

- 하나의 직선으로 분리 가능(선형적으로 분리 가능)
- 단층 퍼셉트론으로 해결 가능

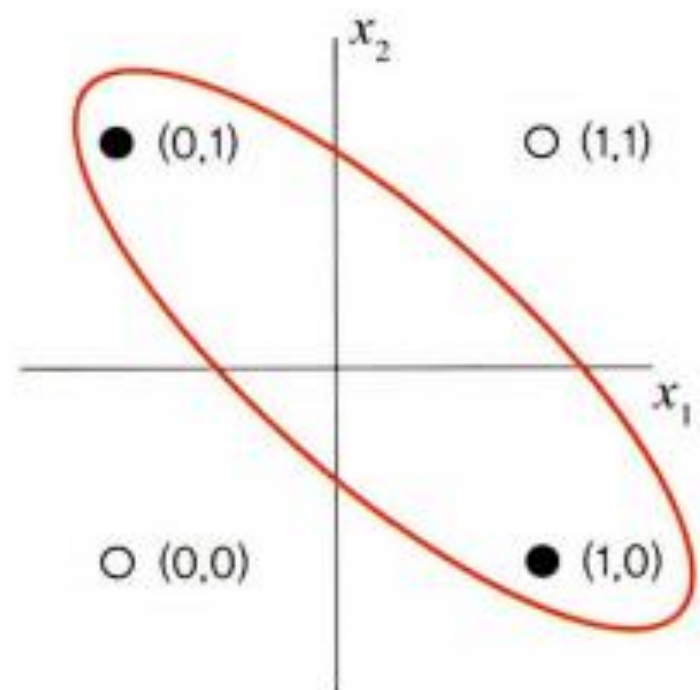
#03 XOR 문제와 단층 퍼셉트론의 한계

그런데 XOR은?

▼ 표 4-3 XOR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

▼ 그림 4-4 XOR 게이트



문제점

하나의 직선으로 분리할 수 없음

-> 선형 분리 불가능

-> 단층 퍼셉트론으로 해결 불가능

#04 다층 퍼셉트론 : 은닉층으로 XOR 해결

단층 퍼셉트론

: 입력 → 출력

- 입력에서 출력으로 직접 연결
- **하나의 선형 결정 경계**
- XOR 해결 X



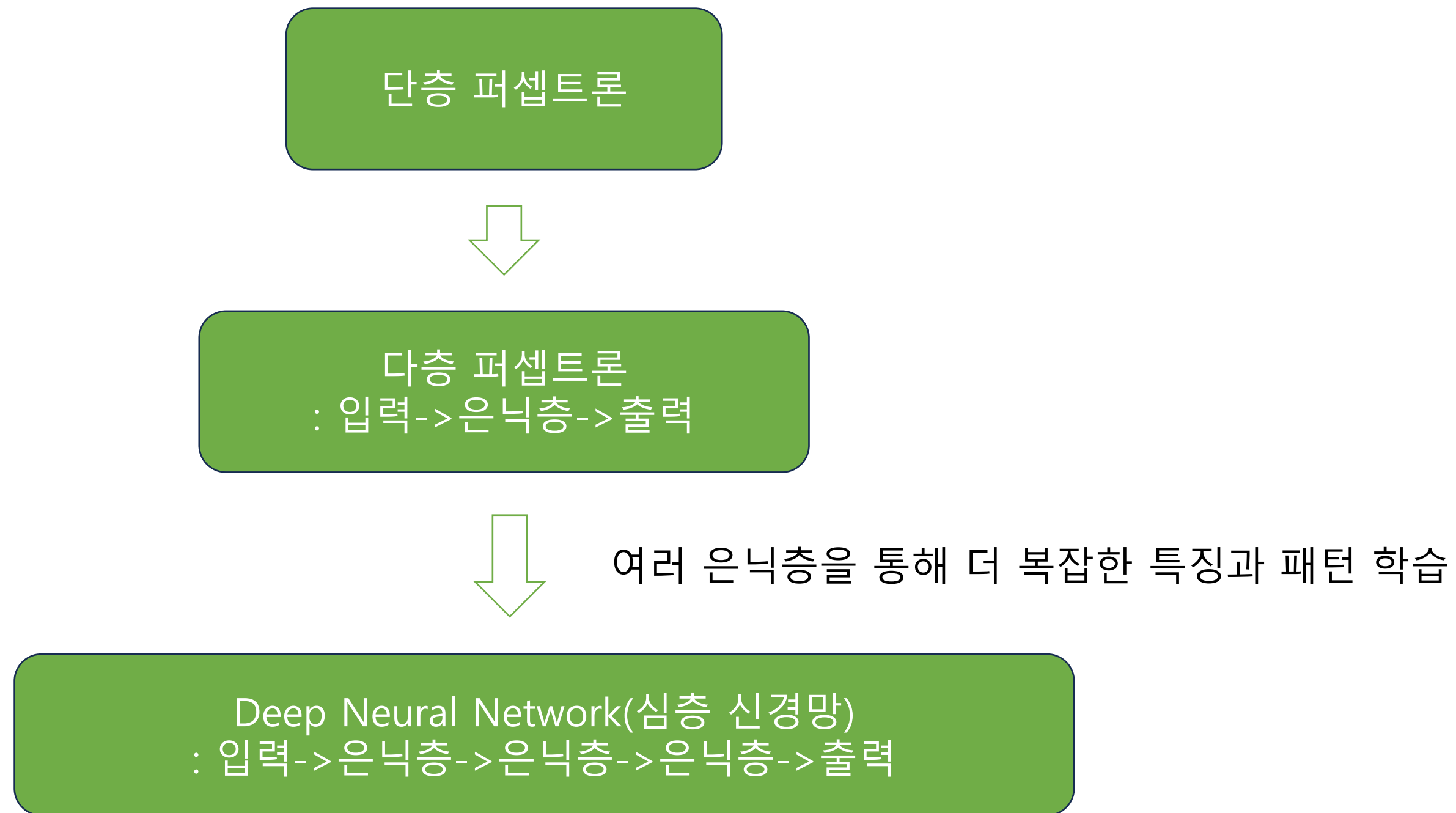
은닉층 추가

다층 퍼셉트론

: 입력 → **은닉층** → 출력

- 은닉층의 여러 뉴런이 서로 다른 경계 생성
- 출력층이 이 결과들을 조합
- **비선형 결정 영역 표현 가능**
- XOR 해결 O

#05 심층 신경망과 딥러닝



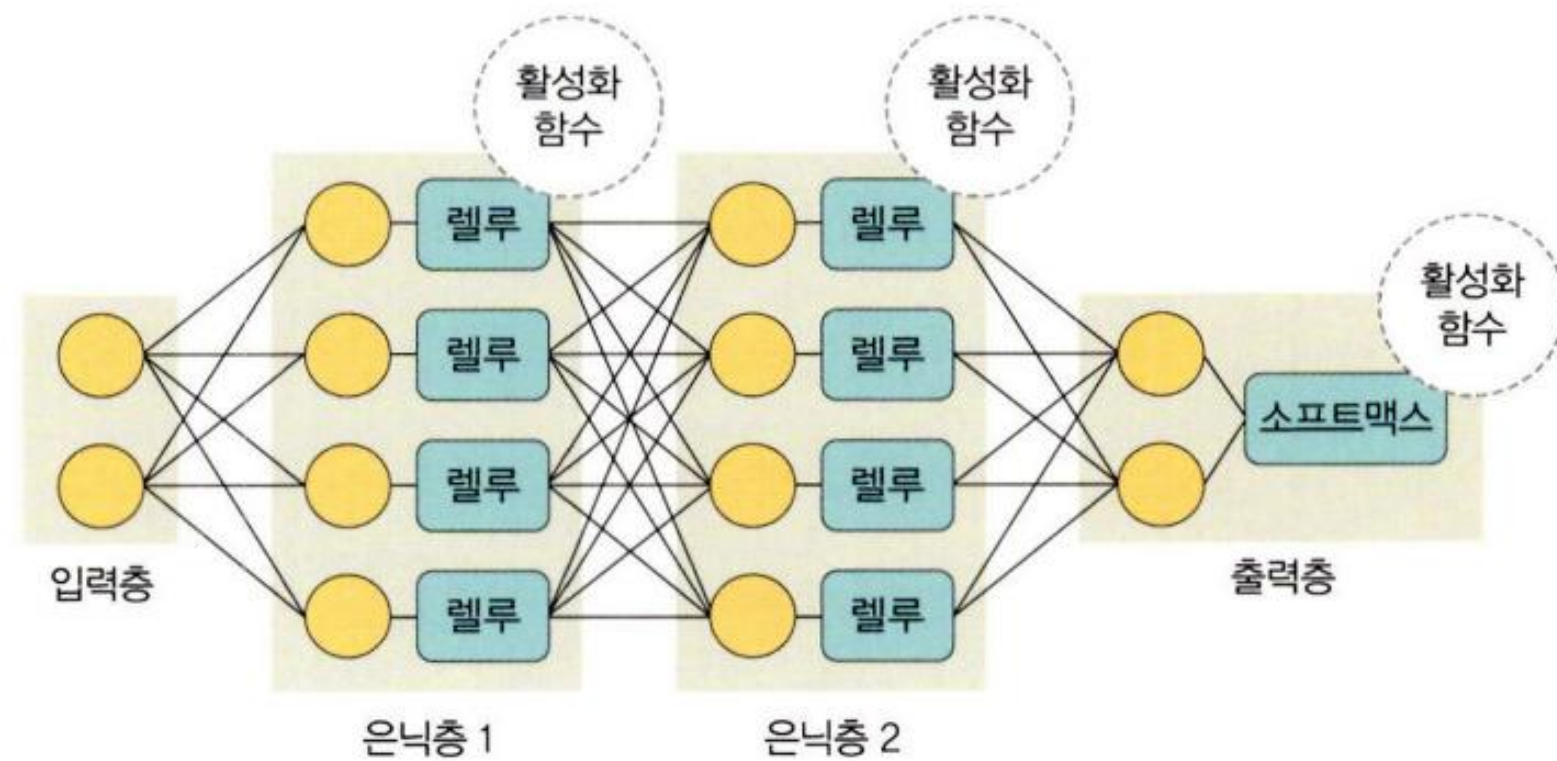
Deep Learning : 심층 신경망을 이용하여 데이터를 학습하는 방법

4.2 딥러닝 구조(1)



#01 딥러닝 구조 : 입력층/은닉층/출력층

▼ 그림 4-5 딥러닝 구조



입력층

→ 원본 데이터를 입력받는 층

은닉층

→ 가중합과 활성화 함수를 통해 입력 정보를 변환하고 특징·패턴 학습

출력층

→ 최종 예측·분류 결과 출력

#02 가중치/편향/활성화 함수

$$z = \sum_i w_i x_i + b$$


$$a = f(z)$$

가중치(w) : 노드와 노드 간 연결 강도
- 입력이 결과에 미치는 영향의 크기

편향(b) : 가중합에 더하는 보정값
- 출력 기준 조절

학습 과정
: 예측 → 손실 계산 → w, b 수정 → 다시 예측

활성화 함수(f) : 가중합을 비선형적으로 변환
- 신경망에 비선형성 부여

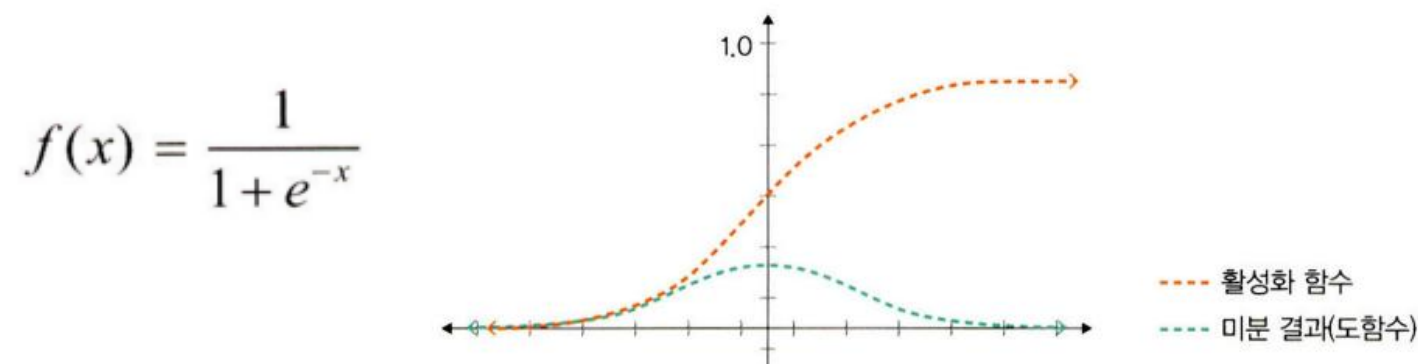
#03 대표적인 활성화 함수 4가지

시그모이드 함수

특징 : 입력값 $\rightarrow$ 0~1 사이 값으로 변환

문제점 : 기울기 소멸 문제

▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과

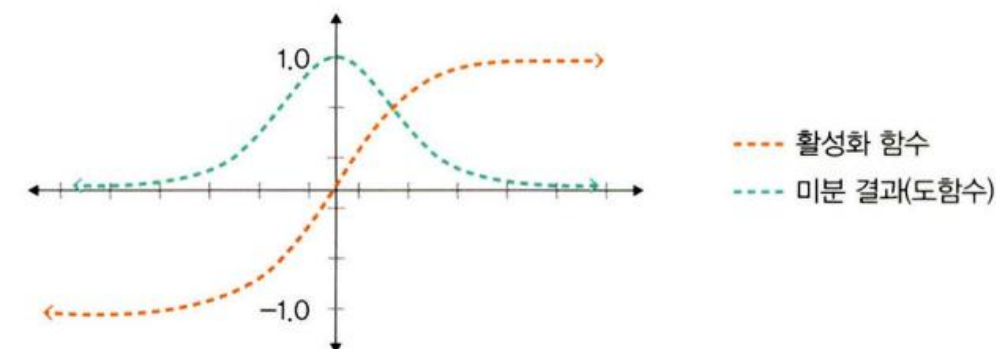


하이퍼볼릭 탄젠트 함수

특징 : 입력값 $\rightarrow$ -1~1 사이 값으로 변환

문제점 : 기울기 소멸 문제

▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과

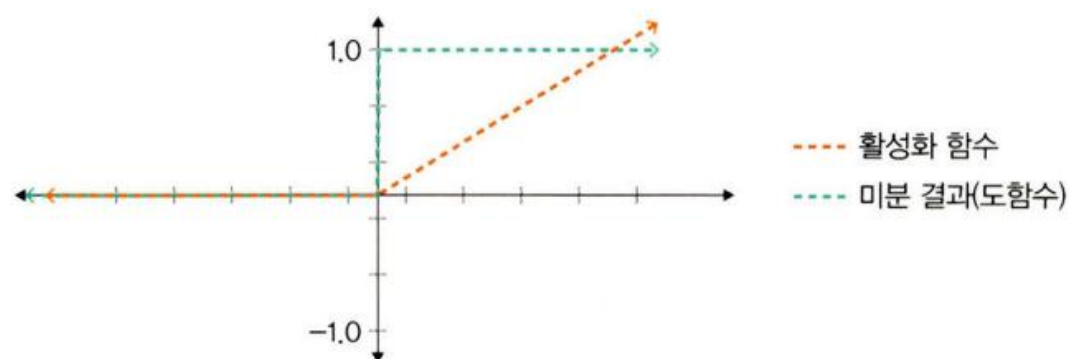


렐루 함수

특징 : 입력음수:0/양수:그대로 출력

문제점 : 음수 영역 기울기 0

▼ 그림 4-10 렐루 활성화 함수와 미분 결과

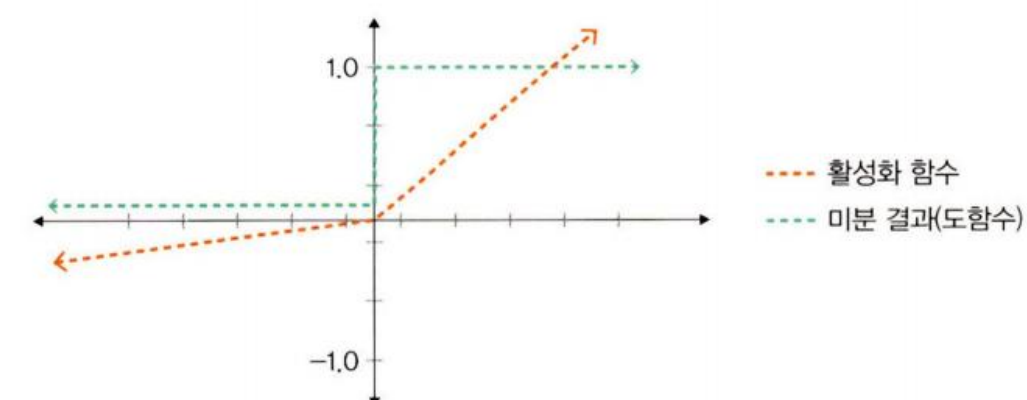


릴리 렐루 함수

특징 : 음수 영역에서도 작은 기울기 유지

$\rightarrow$ ReLU의 문제 오나화

▼ 그림 4-11 릴리 렐루 활성화 함수와 미분 결과



#04 손실 함수 : MSE vs CEE

손실 함수 : 모델의 예측값과 실제값 사이의 차이를 수치화

- 학습 과정 : 예측 → 손실 계산 → Weight·Bias 수정 → 다시 예측

평균 제곱 오차(MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$

- 실제값 ↔ 예측값의 수치 차이
- 회귀 문제
 - Ex) 집값, 온도와 같은 값 예측

크로스 엔트로피 오차(CEE)

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$

- 정답 클래스에 부여한 예측 확률을 평가
- 분류 문제
 - Ex) 실제 정답 : 고양이일 경우,
 - 모델이 고양이에 높은 확률 부여 : 손실 작음
 - 낮은 확률 부여 : 손실 커짐

#4.2.2 딥러닝 학습

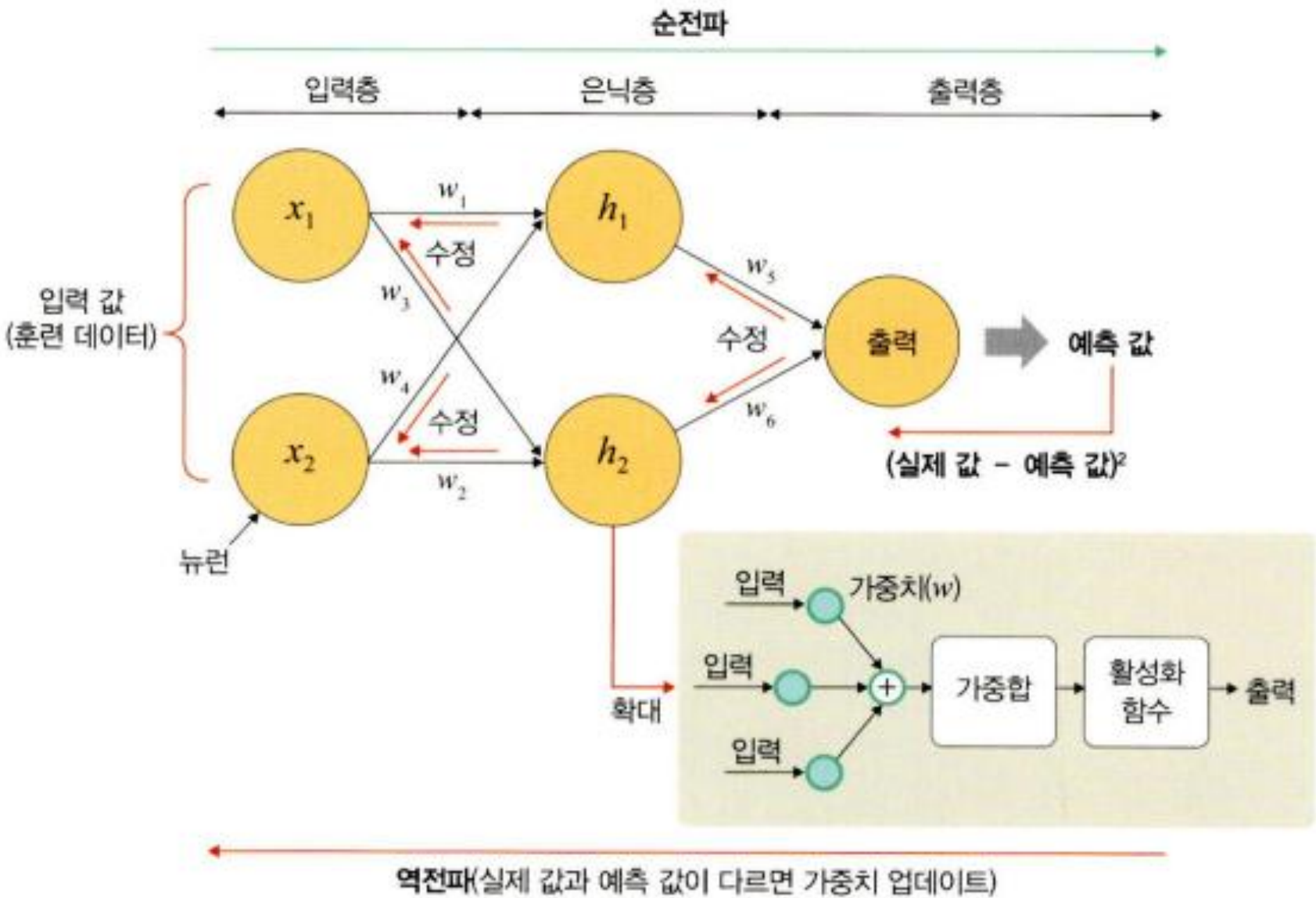
학습 단계

- 1. 순전파 (feedforward)
- 2. 손실 계산
- 3. 역전파(back propaganda)



입력층&은닉층&출력층 노드의 개수, 가중치, 활성화 함수, 손실함수, 학습률 모두 설정

▼ 그림 4-12 순전파와 역전파



순전파

- : 입력층 -> 은닉층 -> 출력층
- : 이전 층의 뉴런에서 수신한 정보에 가중합 및 활성화 함수를 적용하여 다음 층의 뉴런으로 전송한다.

#4.2.2 딥러닝 학습

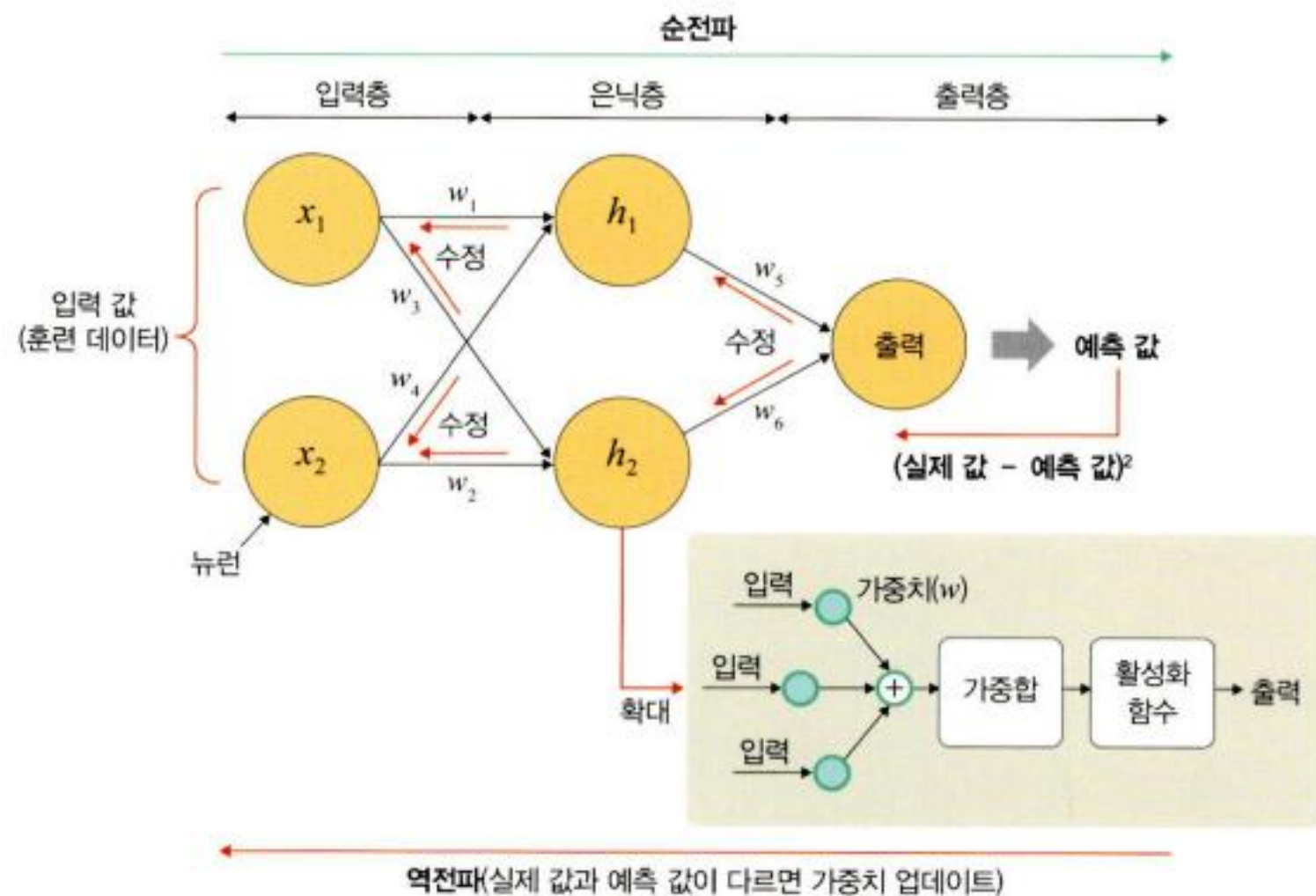
손실 계산

- : 실제 값과 예측 값의 차이를 계산
- : 손실을 최소화하기 위해 역전파를 통해 가중치를 업데이트

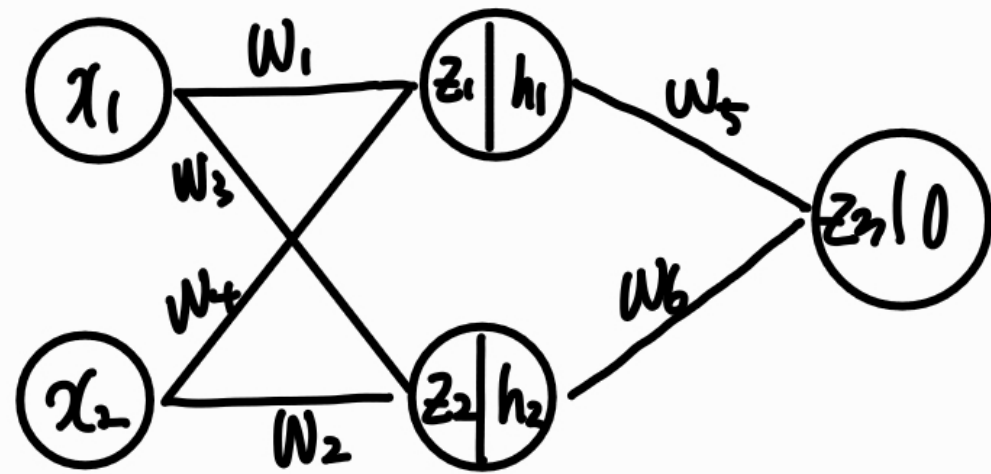
역전파

- : 출력층 → 은닉층 → 입력층
- : 손실 비용은 은닉층의 모든 뉴런으로 전파되며, 이때 각 뉴런의 원래 가중치에 따라 값이 달라진다.
(오차를 각 뉴런의 가중치로 미분한 후 기존 가중치 값에서 뺌)

▼ 그림 4-12 순전파와 역전파



#4.2.2 딥러닝 학습



- 활성화함수 : $\text{sigmoid}(z) = \frac{1}{1+e^{-z}}$
- 손실함수 : $MSE = \frac{1}{n} \sum (y - \hat{y})^2$

1. 순전파 : $z_1 = w_1 x_1 + w_4 x_2$: 가중합
→ $h_1 = \text{sigmoid}(z_1)$: 활성화함수 적용

$$z_3 = w_5 h_1 + w_6 h_2$$

$$\rightarrow 0 = \text{sigmoid}(z_3)$$

2. 손실 계산 : $C = (0 - \hat{0})^2$

3. 역전파 * $w_5 = w_5 - \text{현재값} \times \frac{\partial C}{\partial w_5}$

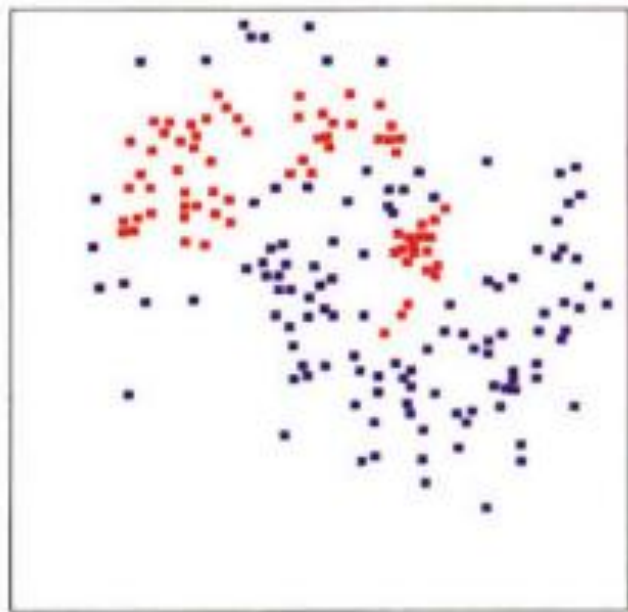
$$\rightarrow \frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial 0} \times \frac{\partial 0}{\partial z_3} \times \frac{\partial z_3}{\partial w_5}$$

* $w_1 \sim w_6$ 구하기

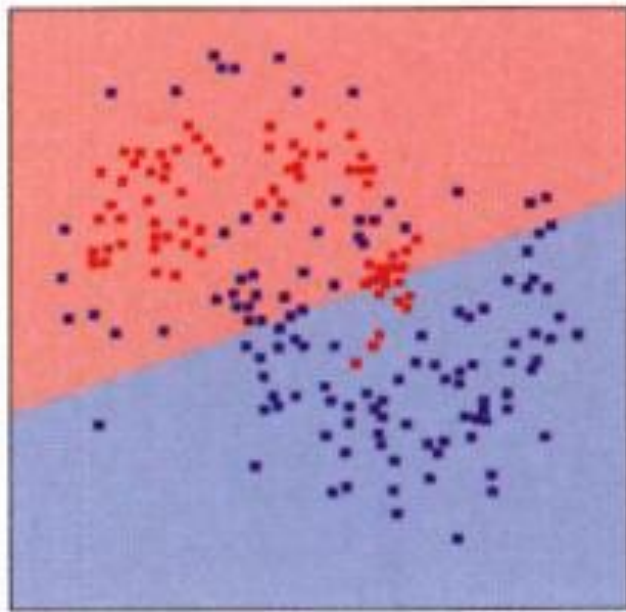
4. Re!!

#4.2.3 딥러닝의 문제점과 해결방안

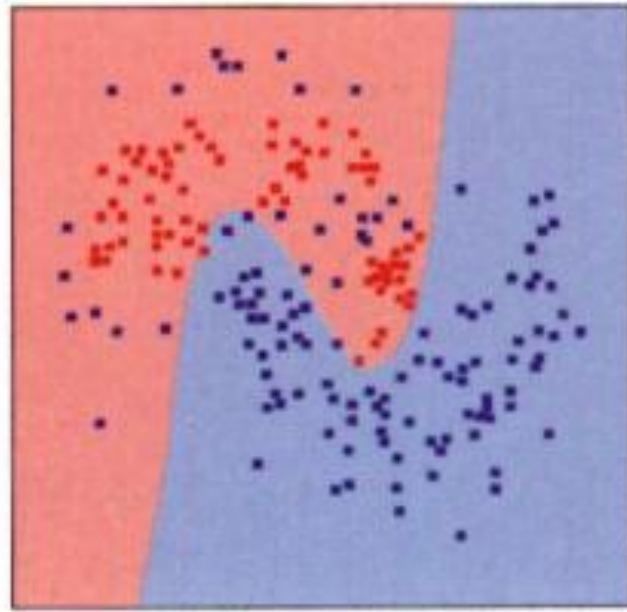
활성화 함수가 적용된 은닉층 개수가 많을수록 데이터 분류가 잘된다.



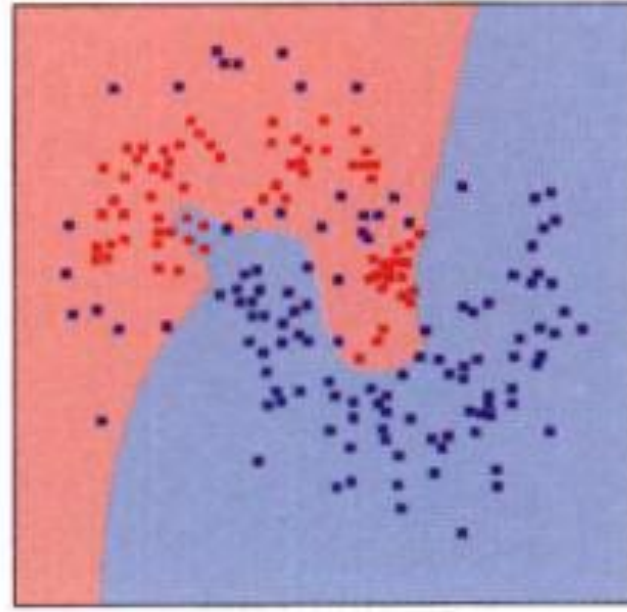
데이터



로지스틱 회귀



은닉층 세 개



은닉층 50개

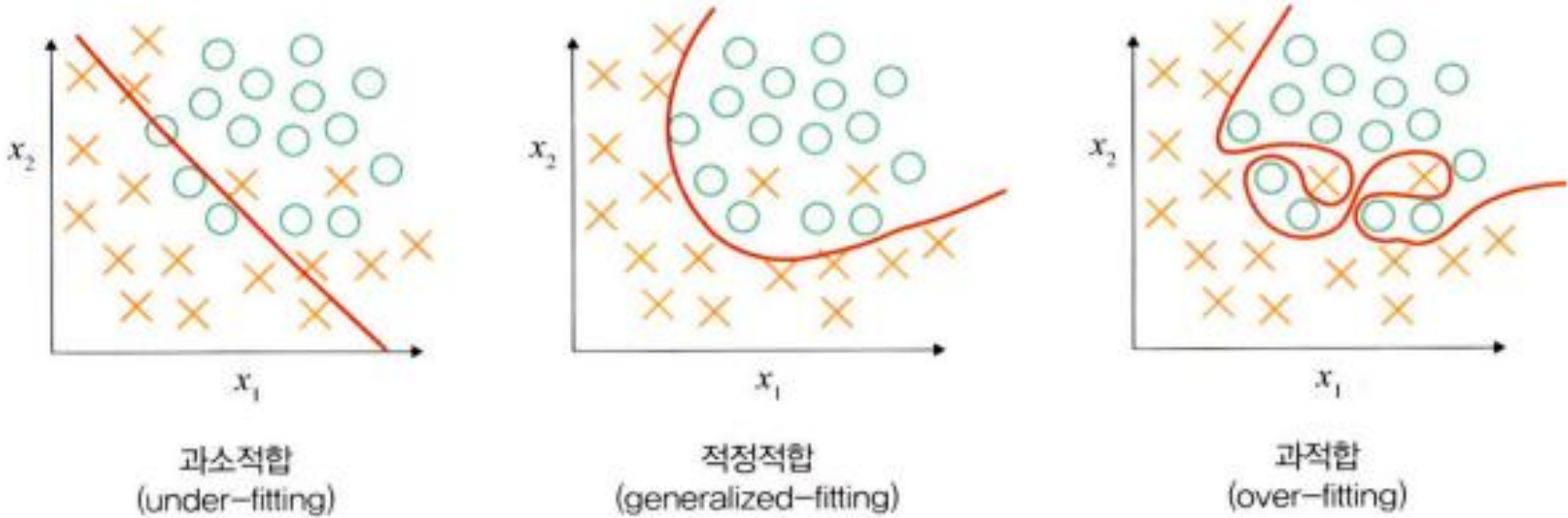
#4.2.3 딥러닝의 문제점과 해결방안

문제 – ① 과적합 (over-fitting)

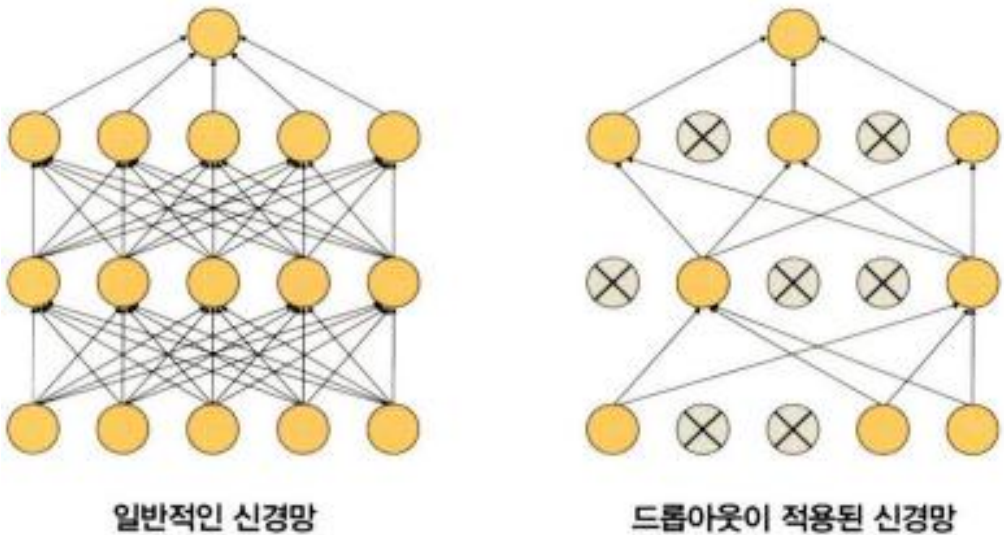
: 훈련 데이터를 과하게 학습하여 발생하는 문제

해결 방법 ->

드롭 아웃



- : 학습 과정 중 임의로 살릴 확률을 p로 두어 층마다 일부 노드들을 학습에서 제외한다.
- : 그러나 추후 테스트 단계에서는 모든 노드를 살린다.



```
class DropoutModel(torch.nn.Module):
    def __init__(self):
        super(DropoutModel, self).__init__()
        self.layer1 = torch.nn.Linear(784, 1200)
        self.dropout1 = torch.nn.Dropout(0.5) ----- 50%의 노드를 무작위로 선택하여 사용하지 않겠다는 의미
        self.layer2 = torch.nn.Linear(1200, 1200)
        self.dropout2 = torch.nn.Dropout(0.5)
        self.layer3 = torch.nn.Linear(1200, 10)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.dropout1(x)
        x = F.relu(self.layer2(x))
        x = self.dropout2(x)
        return self.layer3(x)
```

#4.2.3 딥러닝의 문제점과 해결방안

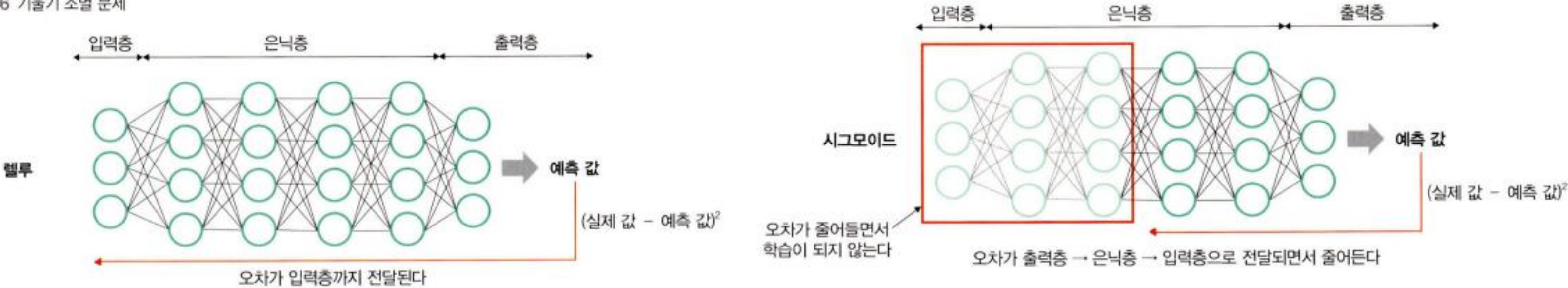
문제 – ② 기울기 소멸

: 출력층에서 은닉층으로 전달되는 과정에서 오차가 줄어들어 기울기가 소멸되어 학습되는 양이 0이 되는 문제

해결 방법 ->

렐루 활성화 함수 (시그모이드나 하이퍼볼릭 탄젠트 대신)

▼ 그림 4-16 기울기 소멸 문제



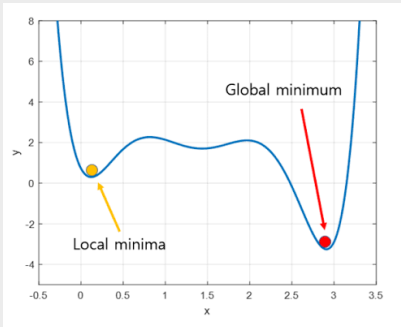
#4.2.3 딥러닝의 문제점과 해결방안

문제 – ③ 성능 저하

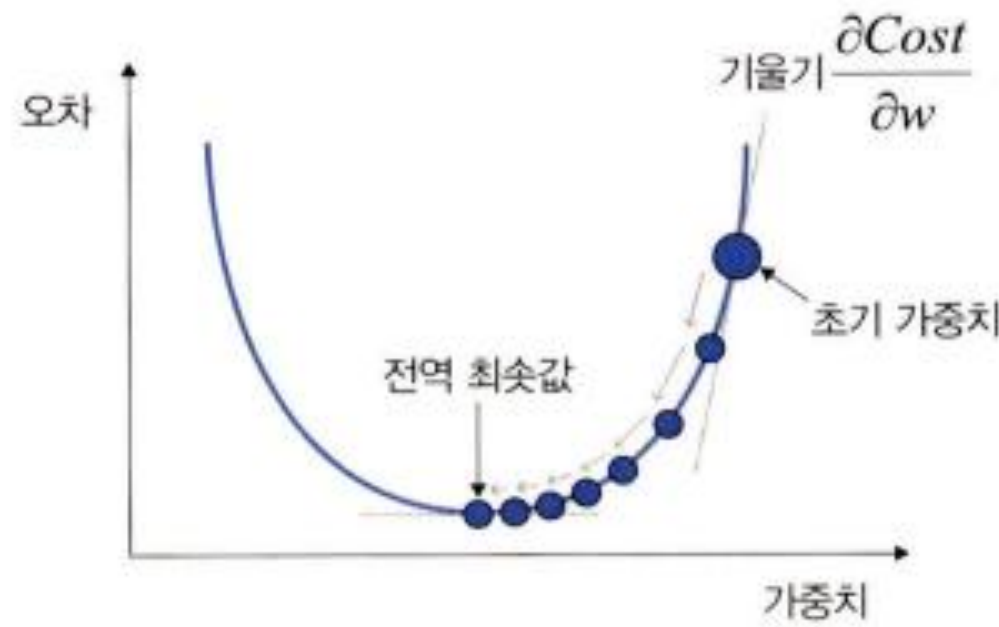
: 경사 하강법에서 한 번 이동(Step)하기 위한 계산 양이 늘어나 속도가 느려지는 등의 성능 저하가 발생한다.

경사 하강법

손실 함수의 비용이 최소가 되는 지점(Global Minimum)을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시킨다. ($w^* = w - \alpha \frac{\partial E}{\partial w}$)



♥ 그림 4-17 경사 하강법



해결 방법 ->

확률적 경사 하강법 & 미니 배치 경사 하강법

#4.2.3 딥러닝의 문제점과 해결방안

문제 – ③ 성능 저하

배치 경사 하강법(Batch Gradient Descent, BGD)

: 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트 하는 방법

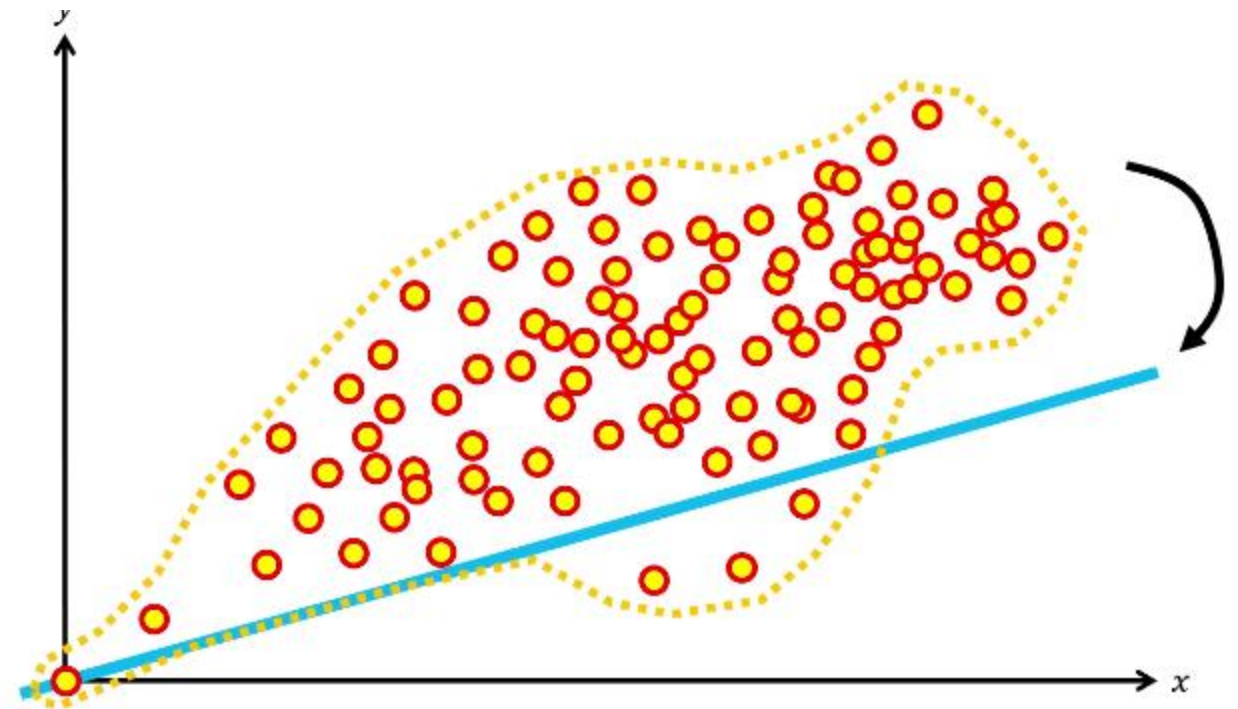
: 모든 훈련 데이터셋을 사용하므로 학습이 오래 걸리는 단점

$$W = W - a \nabla J(W, b)$$

(a : 학습률, J : 손실 함수)



$$W = W - a \nabla J(W, b)$$



[출처 : <https://inner-game.tistory.com/711>]

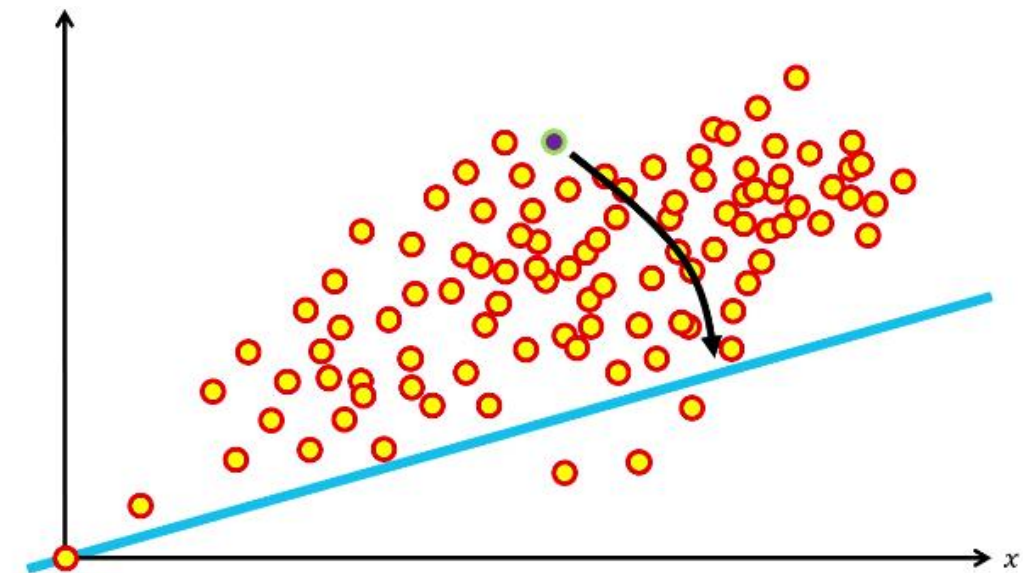
#4.2.3 딥러닝의 문제점과 해결방안

문제 – ③ 성능 저하

확률적 경사 하강법(Stochastic Gradient Descent, SGD)

: 임의로 선택한 데이터에 가중치를 업데이트 한다.

: 배치 경사 하강법보다 효율적이지만, 데이터 하나하나에 따라 변동폭이 크므로 불안정하다.



[출처 : <https://inner-game.tistory.com/711>]

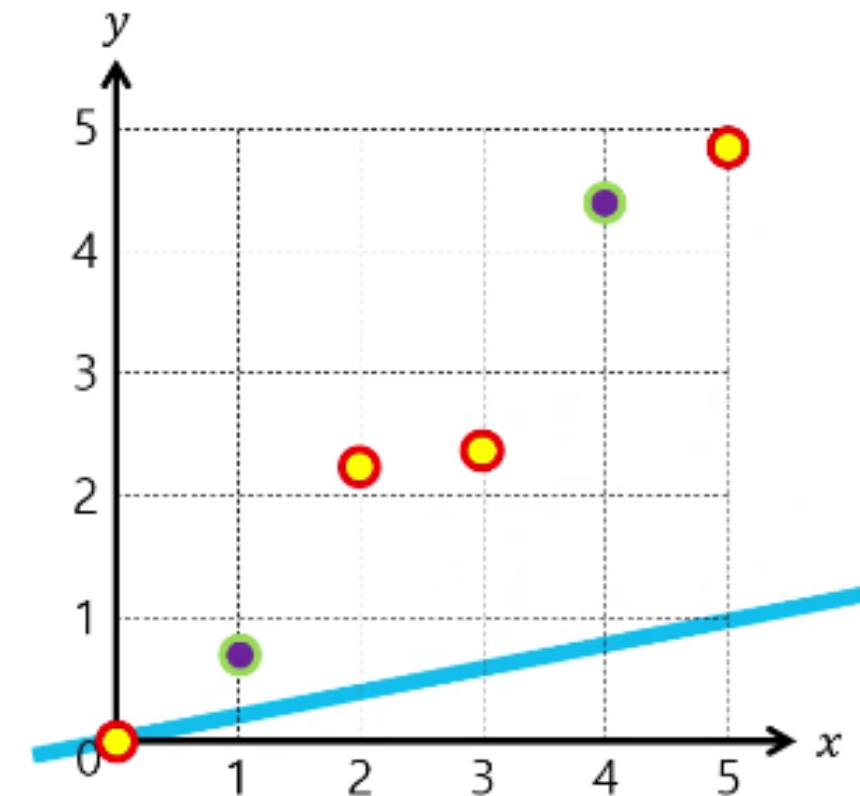
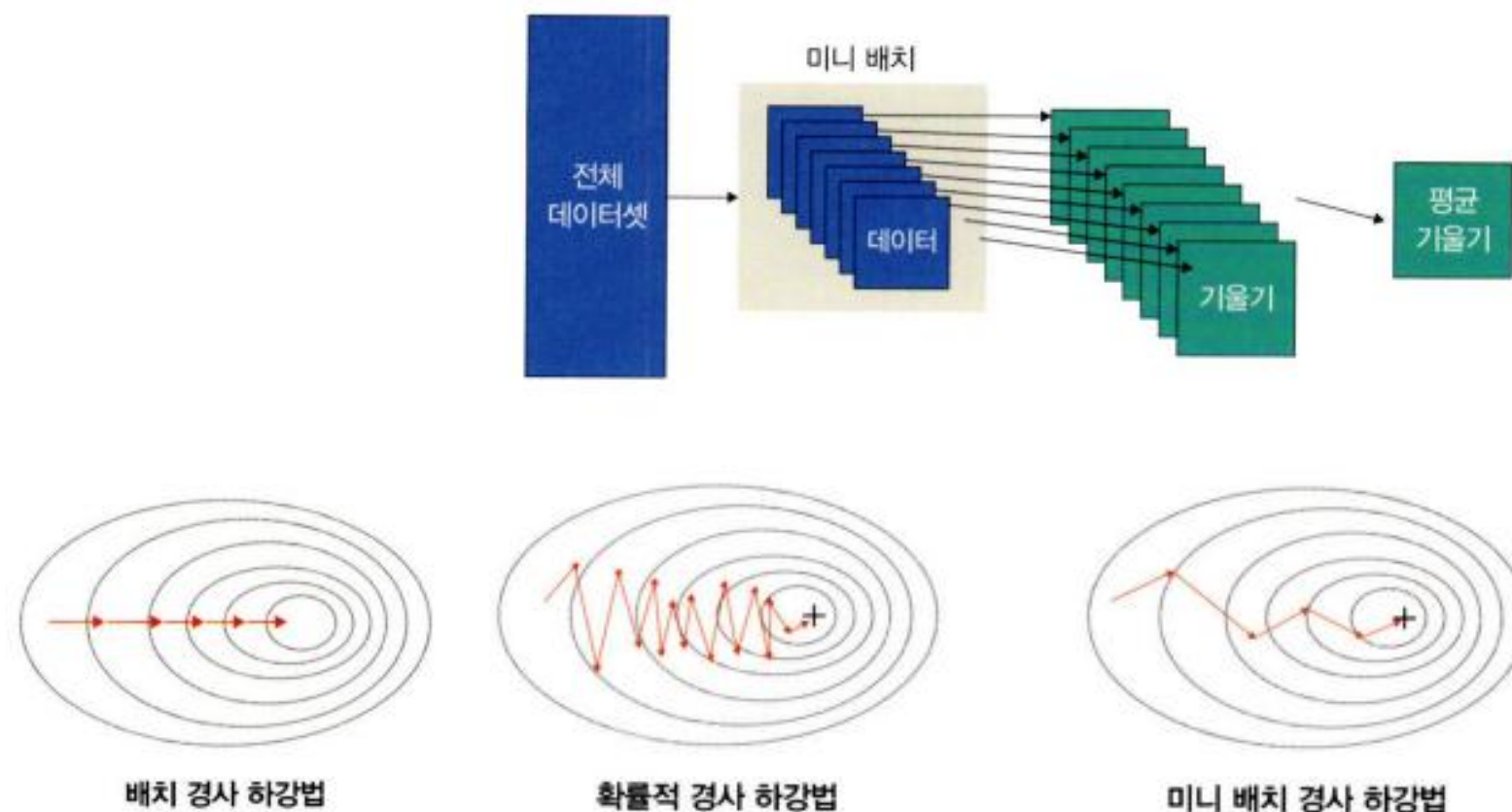
#4.2.3 딥러닝의 문제점과 해결방안

문제 – ③ 성능 저하

미니 배치 경사 하강법(mini-batch gradient descent)

: 전체 데이터셋을 미니 배치 여러 개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트해서 학습하는 방법이다.

: BGD보다 빠르며, SGD보다 안정적이라는 장점이 있기 때문에 실제로 가장 많이 사용한다.

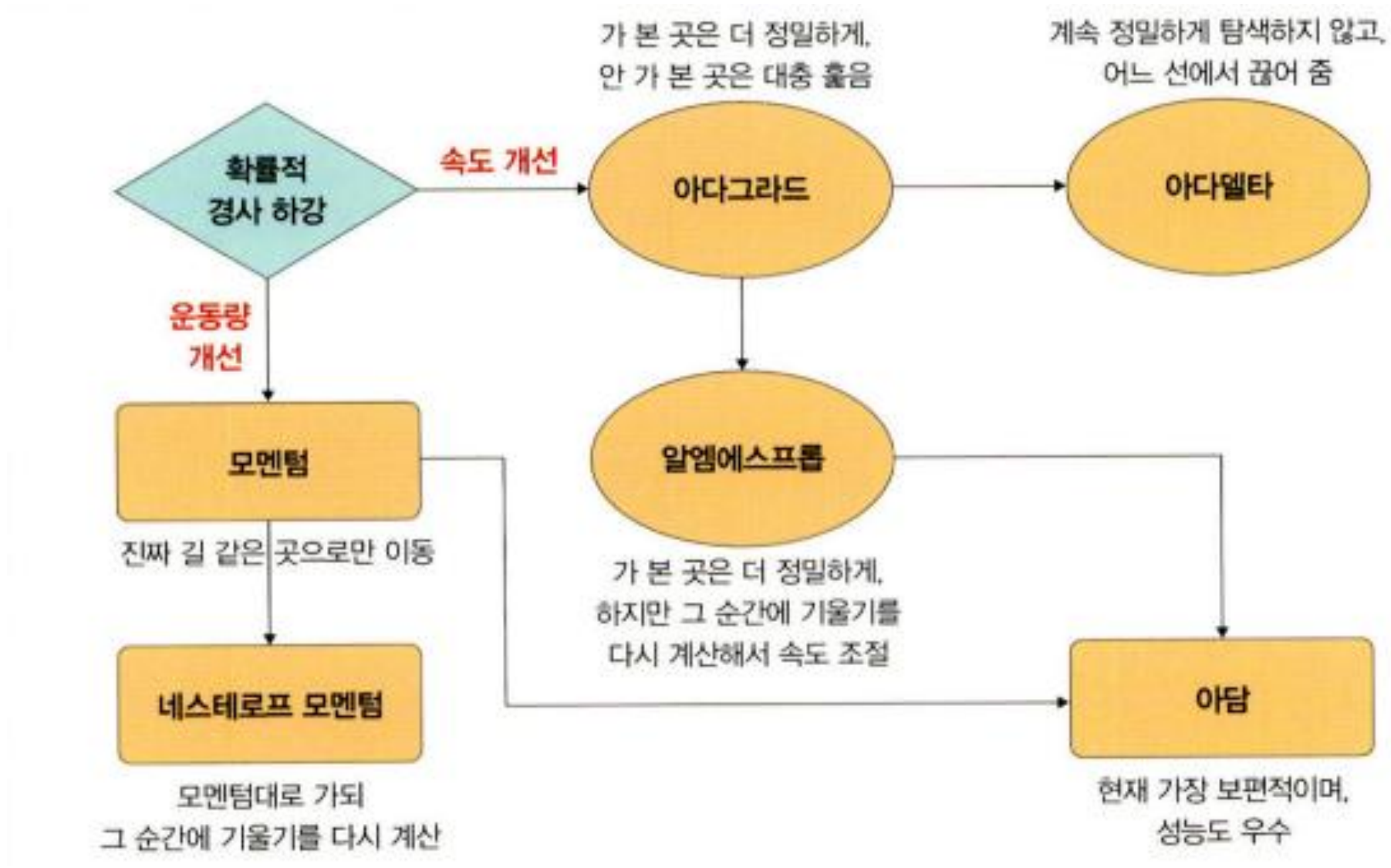


[출처 : <https://inner-game.tistory.com/711>]

#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

: SGD의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조정



속도 조정

- 아다그라드
- 아다델타
- 알엠에스프롬

운동량 조정

- 모멘텀
- 네스테로프 모멘텀

혼용

- 아담

#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

속도 조정 – 아다그리드 (Adagrad, Adaptive gradient)

- : 많이 변화한 변수들은 학습률을 작게, 많이 변화하지 않은 변수들의 학습률을 크게한다.
- : G 값은 이전 G값의 누전(기울기 크기의 누적)으로, G값이 커지면 학습률이 작아진다.
- : G값이 커져 기울기가 0에 수렴할 때 정체하는 문제가 있어, 이 대신 RMSProp를 사용한다.

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$
$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

```
optimizer = torch.optim. Adagrad(model.parameters(), lr=0.01)
```

#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

속도 조정 – 아다델타 (Adadelta, Adaptive delta)

: Adagrad에서 G값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장

: Adagrad의 수식에서 학습률을 D함수(가중치의 변화량 크기를 누적한 값)로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요하지 않다.

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\epsilon}}{\sqrt{G(i)+\epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

```
optimizer = torch.optim. Adadelta(model.parameters(), lr=1.0)
```

#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

속도 조정 – 알엠에스프롭 (RMSProp)

: Adagrad에서 G값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장

: G함수에 감마를 추가하여, 이를 통해 학습률 크기를 비율로 조정할 수 있다.

즉 감마가 커지면 이전 학습률을 더 중요히 여기는 것이고, 감마가 작아지면 현재 학습률을 중요시한다.

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

```
optimizer = torch.optim.RMSprop (model.parameters(), lr=0.01)
```

#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

운동량 조정-모멘텀 (Momentum)

- : 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정한다.
- : 이로 인해 SGD의 지그재그 현상이 줄어들고, 이전 이동 값을 고려하여 일정 비율만큼 다음 값을 결정하므로 관성 효과를 얻을 수 있다.
- : SGD 모멘텀은 기울기를 속도로 대체하여 사용하는 방식으로 이전 속도의 일부를 반영하는, 즉 이전에 학습했던 속도와 현재 기울기를 반영해서 가중치를 구한다.

$$w(i+1) = w(i) - v(i)$$
$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$$

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

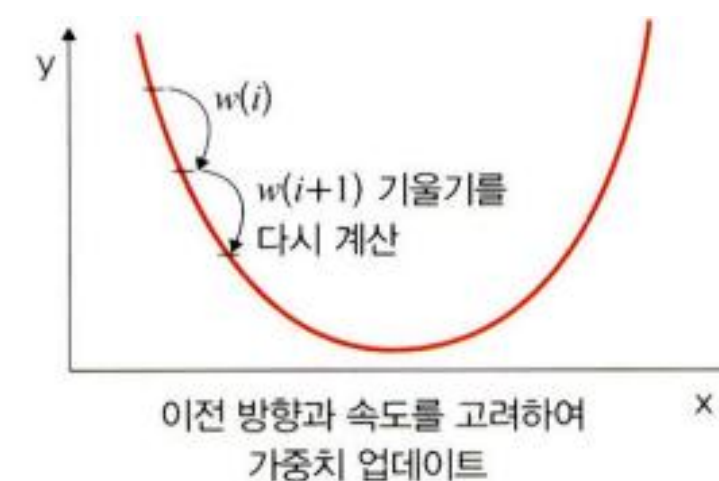
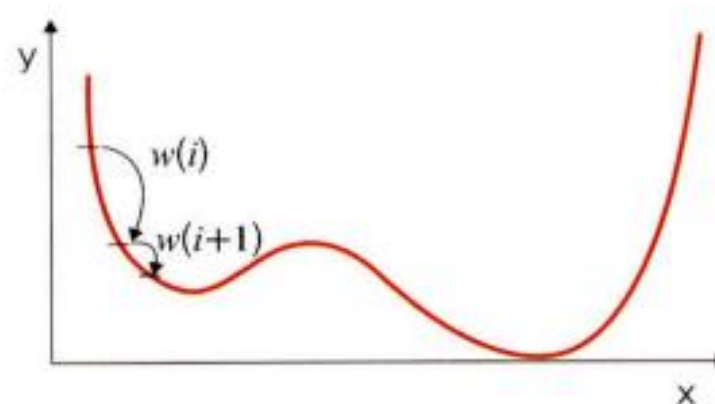
#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

운동량 조정 – 네스테로프 모멘텀 (Nesterov Accelerated Gradient, NAG)

- : 모멘텀으로 절반 정도 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정하기 때문에, 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있는 단점을 극복할 수 있다.
- : 모멘텀과 비슷하지만 속도를 구하는 과정에서 이전에 학습했던 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 더해서 가중치를 구한다.

$$w(i+1) = w(i) - v(i)$$
$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$$



```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, Nesterov=True)
```

#4.2.3 딥러닝의 문제점과 해결방안

옵티마이저

혼용 – 아담 (Adam, Adaptive Moment Estimation)

- : RMSProp과 모멘텀을 결합
- : RMSProp의 기울기 제곱을 지수 평균한 값과 모멘텀의 특징인 $v(i)$ 를 수식에 활용한다.
- : 가장 많이 활용되는 옵티마이저 방법이다.

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} v(i)$$
$$G(i) = \gamma_2 G(i-1) + (1 - \gamma_2) (\nabla E(w(i)))^2$$
$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

#4.2.4 딥러닝을 사용할 때 이점

특성 추출 & 벡터데이터

: 기존에는 데이터의 분야에 대한 전문 지식을 기반으로 데이터를 벡터로 변환하는 특성 추출(feature extraction)이 필요했다.

: 그러나 ‘벡터데이터+딥러닝의 은닉층’을 통해 특성 추출이 알고리즘에 통합되며 성능 향상이 일어났다.

다르게 말하자면, 확보된 데이터가 적다면 머신러닝을 사용하는 것이 좋다.

4.3 딥러닝 알고리즘



#4.3 딥러닝 알고리즘

딥러닝 알고리즘의 분류

딥러닝 알고리즘은 심층 신경망을 사용한다는 공통점이 있음
머신 러닝 알고리즘처럼 목적에 따라 다음과 같이 분류됨

- 심층 신경망 (DNN)
- 합성곱 신경망 (CNN)
- 순환 신경망 (RNN)
- 제한된 볼츠만 머신 (RBM)
- 심층 신뢰 신경망 (DBN)

#4.3.1 심층 신경망 (DNN)

심층 신경망이란 ?

- 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망
- 머신 러닝에서는 비선형 분류를 위해 여러 트릭 (trick) 을 사용했음
- 심층 신경망은 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류가 가능

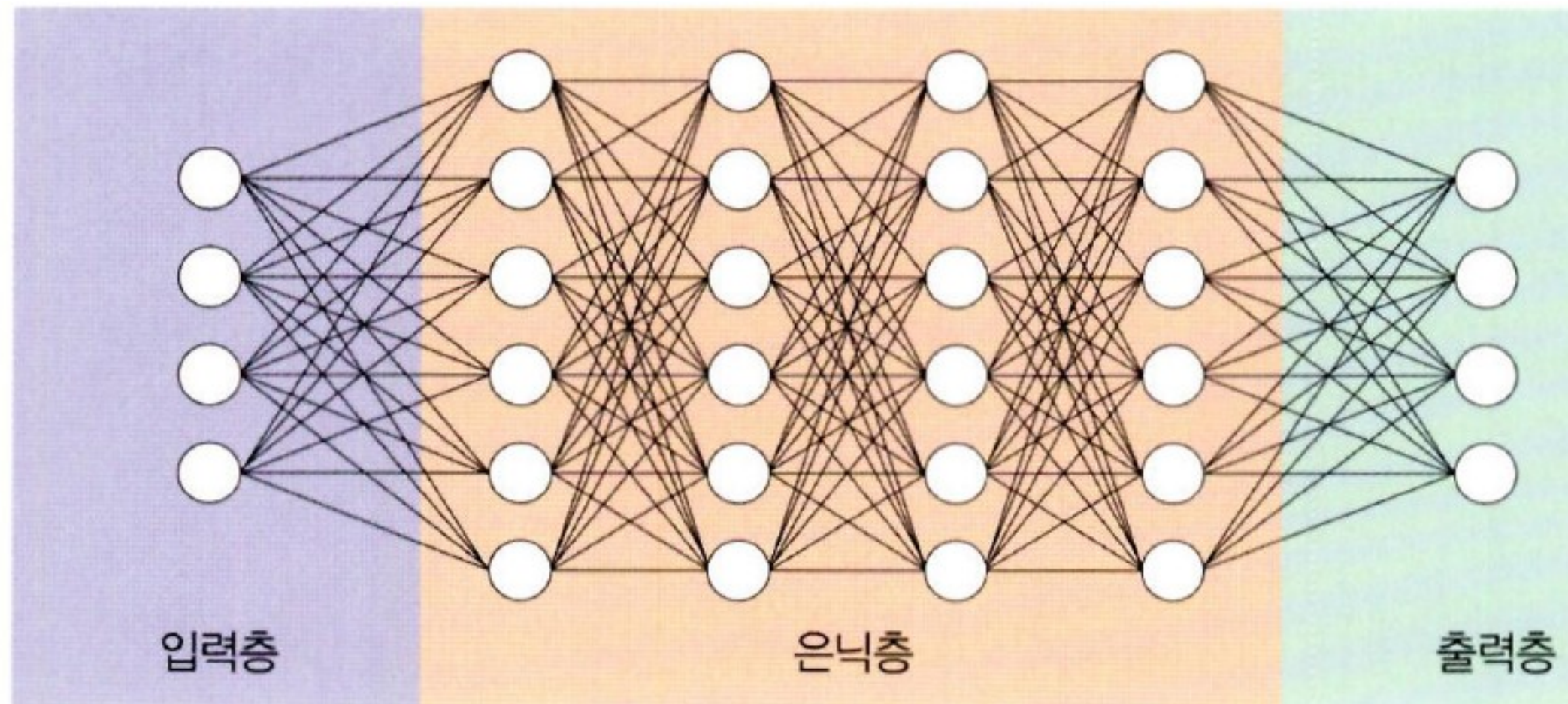


그림 4-24 심층 신경망

#4.3.1 심층 신경망 (DNN)

장점과 단점

장점

- 다수의 은닉층을 두었기 때문에 다양한 비선형적 관계를 학습할 수 있음

단점

- 학습을 위한 연산량이 많음
- 기울기 소멸 문제 등이 발생할 수 있음

해결 방안 : 앞서 설명한 드롭아웃, 렐루 함수, 배치 정규화 등을 적용

#4.3.2 합성곱 신경망 (CNN)

합성곱 신경망이란 ?

- 합성곱층 (convolutional layer) 과 풀링층 (pooling layer) 을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용

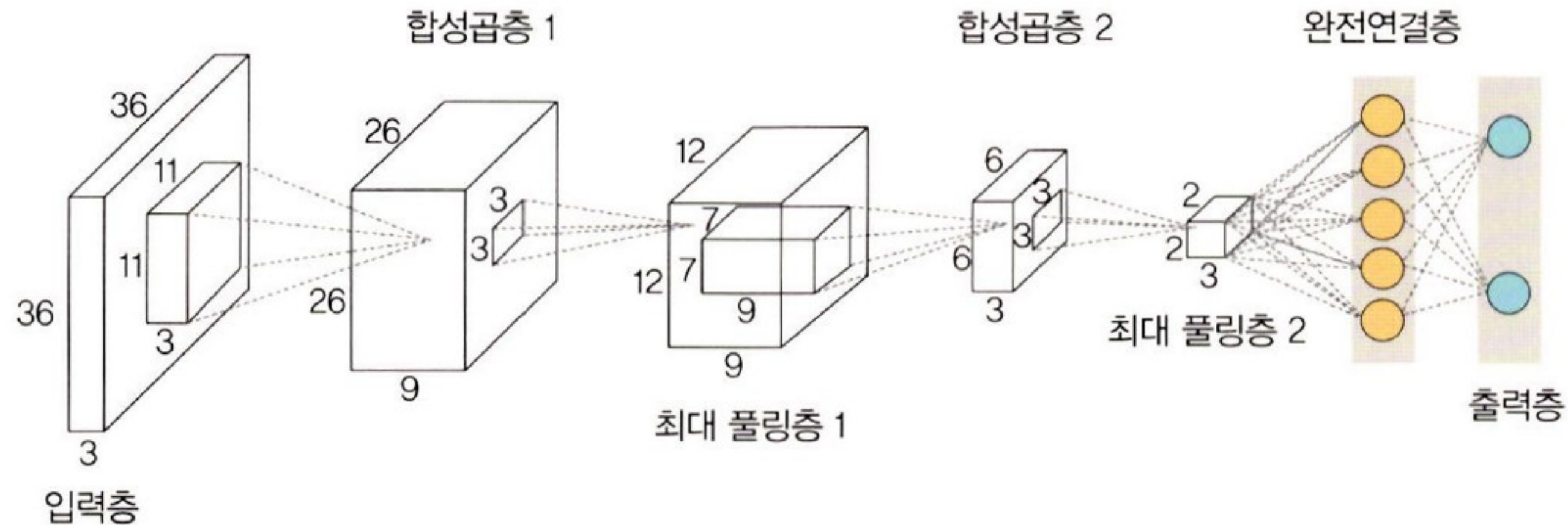


그림 4-25 합성곱 신경망

#4.3.2 합성곱 신경망 (CNN)

기존 신경망과의 차별성

- 각 층의 입출력 형상을 유지함
- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징을 추출하고 학습함
- 추출한 이미지의 특징을 모으고 강화하는 풀링층이 있음
- 필터를 공유 파라미터로 사용하기 때문에 일반 인공 신경망과 비교하여 학습 파라미터가 매우 적음

#4.3.2 합성곱 신경망 (CNN)

대표적인 합성곱 신경망

- 합성곱 신경망은 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 특히 유용

대표적인 합성곱 신경망

- LeNet-5, AlexNet

층을 더 깊게 쌓은 신경망

- VGG, GoogLeNet, ResNet

합성곱 신경망은 5~6 장에서 자세히 다룸

#4.3.3 순환 신경망 (RNN)

순환 신경망이란 ?

- 시계열 데이터 (음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
- '순환 (recurrent)' 은 자기 자신을 참조한다는 것
→ 현재 결과가 이전 결과와 연관이 있다는 의미

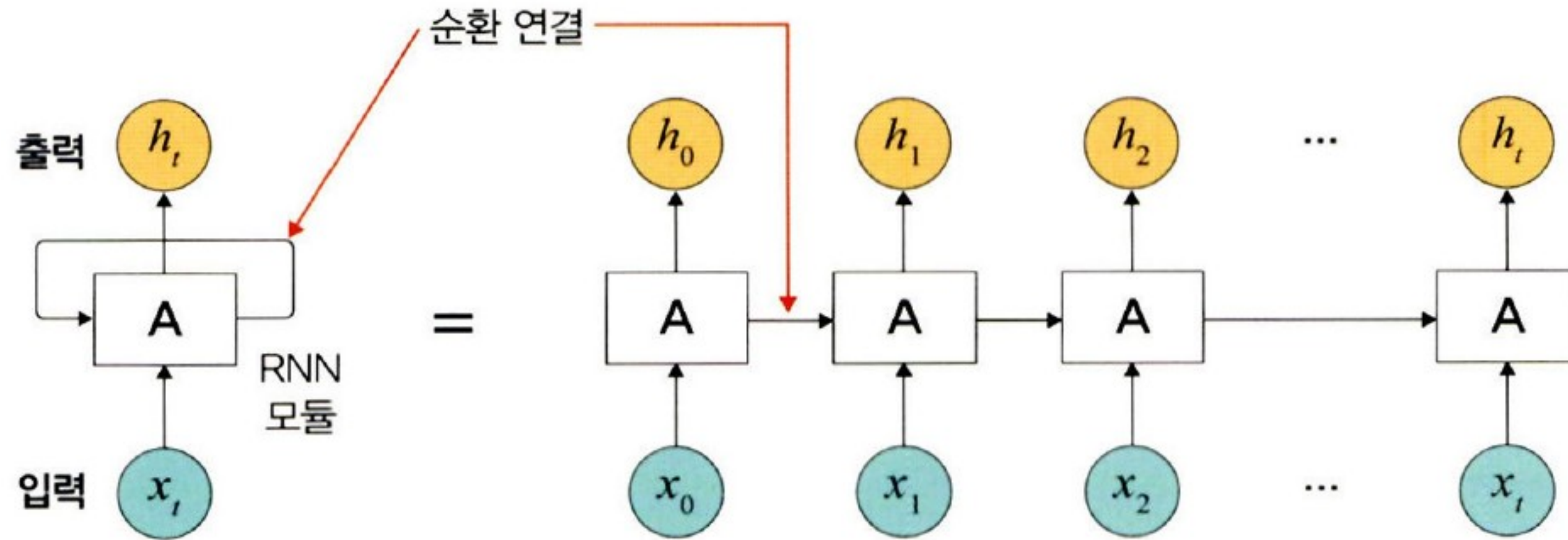


그림 4-26 순환 신경망

#4.3.3 순환 신경망 (RNN)

순환 신경망의 특징

- 시간성 (temporal property) 을 가진 데이터가 많음
- 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
- 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적임
- 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음

#4.3.3 순환 신경망 (RNN)

한계와 LSTM, 그리고 활용 분야

한계

- 기울기 소멸 문제 (vanishing gradient problem) 로 학습이 제대로 되지 않는 문제가 있음

해결 방안

- 메모리 개념을 도입한 LSTM(Long-Short Term Memory) 이 많이 사용되고 있음

활용 분야 : 자연어 처리 분야와 궁합이 맞음

- 언어 모델링, 텍스트 생성, 자동 번역 (기계 번역), 음성 인식, 이미지 캡션 생성 등

순환 신경망은 7 장 '시계열 분석' 에서 자세히 다룸

#4.3.4 제한된 볼츠만 머신 (RBM)

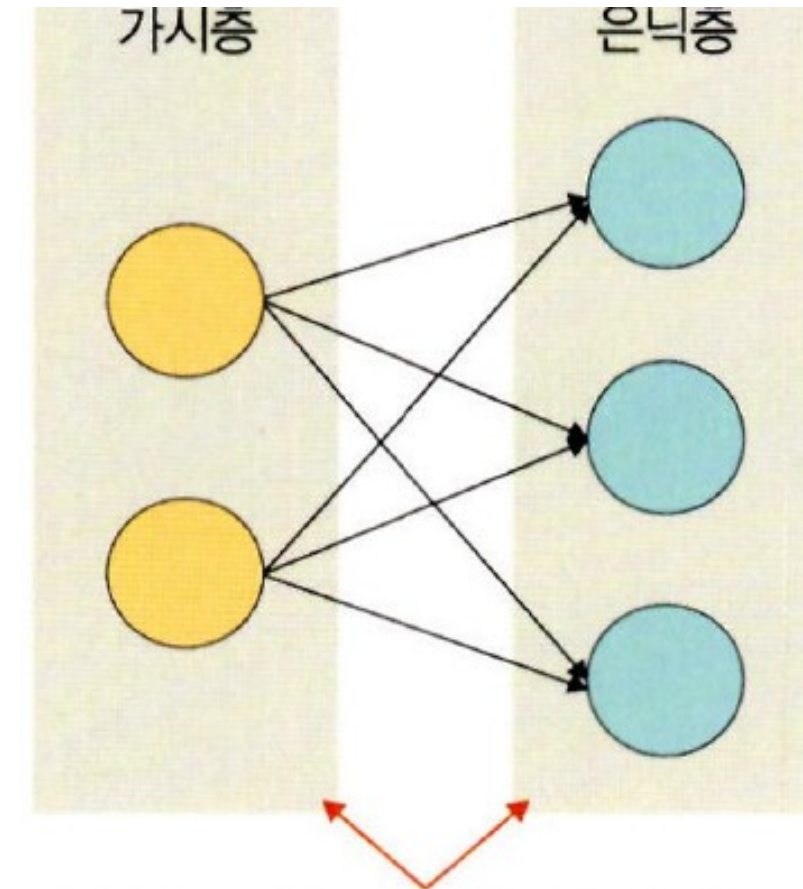
볼츠만 머신과 제한된 볼츠만 머신

볼츠만 머신 (Boltzmann machine)

- 가시층 (visible layer) 과 은닉층 (hidden layer) 으로 구성된 모델

제한된 볼츠만 머신 (Restricted Boltzmann Machine, RBM)

- 가시층은 은닉층과만 연결됨
- 가시층과 가시층, 은닉층과 은닉층 사이에는 연결이 없음



가시층 간, 은닉층 간에 연결은 없다

그림 4-27 제한된 볼츠만 머신

#4.3.4 제한된 볼츠만 머신 (RBM)

제한된 볼츠만 머신의 특징

- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링 (collaborative filtering), 특성 값 학습 (feature learning), 주제 모델링 (topic modelling) 에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망 (DBN) 의 요소로 활용됨

딥러닝에서 많이 사용되는 알고리즘은 CNN 과 RNN

제한된 볼츠만 머신과 심층 신뢰 신경망은 상대적으로 많이 사용하지 않음

#4.3.5 심층 신뢰 신경망 (DBN)

심층 신뢰 신경망이란 ?

- 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망
- 레이블이 없는 데이터에 대한 비지도 학습이 가능
- 부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 유용

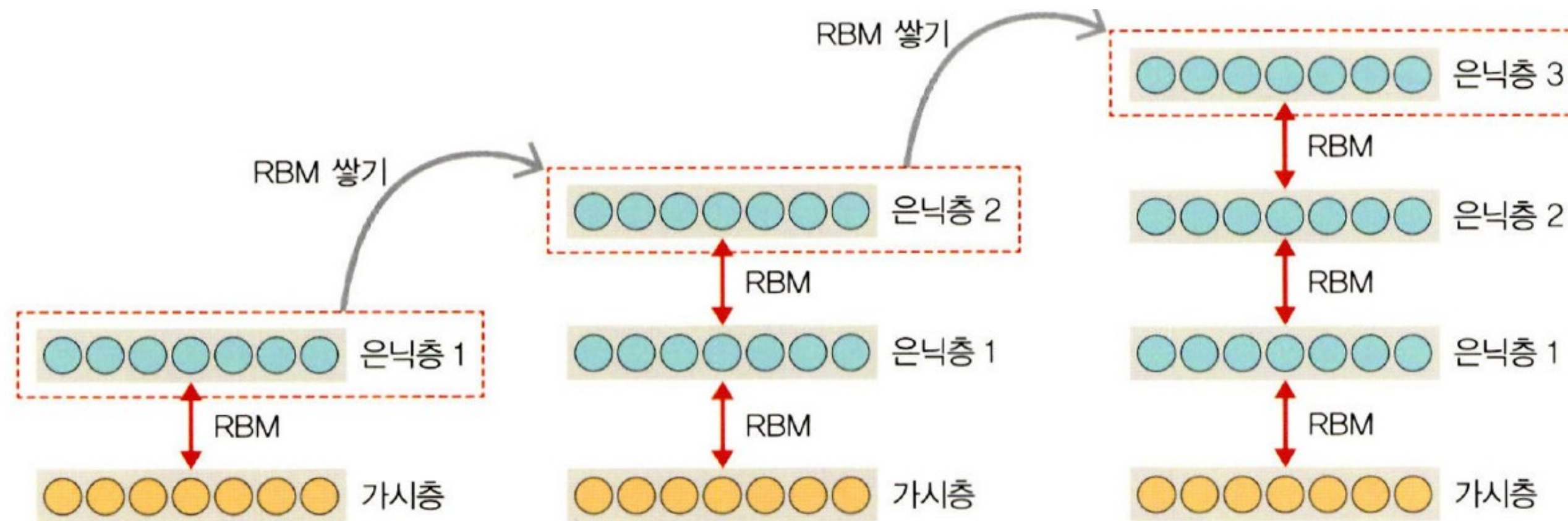


그림 4-28 심층 신뢰 신경망

#4.3.5 심층 신뢰 신경망 (DBN)

학습 절차와 특징

학습 절차

1. 가시층과 은닉층 1에 제한된 볼츠만 머신을 사전 훈련함
2. 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련함
3. 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN을 완성함

심층 신뢰 신경망의 특징

- 순차적으로 학습시켜 가면서 계층적 구조를 생성함
- 비지도 학습으로 학습함
- 위로 올라갈수록 추상적 특성을 추출함
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용함

#4.4 우리는 무엇을 배워야 할까 ?

머신 러닝과 딥러닝, 무엇을 선택할까 ?

머신 러닝을 학습할지 딥러닝을 학습할지는 주어진 데이터를 활용하여 어떤 결과를 얻고 싶은지에 따라 다름

- 간단한 선형 회귀 분류로 원하는 값을 도출할 수 있다면
→ 머신 러닝만으로도 충분
- 복잡한 비선형 데이터에 대한 분류 및 예측을 도출하고 싶다면
→ 딥러닝으로 학습해야 함

머신 러닝과 딥러닝은 우리가 얻고 싶은 결과를 도출하기 위한 도구 (연장) 일 뿐

어떤 도구가 더 좋은 성능과 효과를 낼지는 데이터를 수집하고 분석하는 사람만 알 수 있음